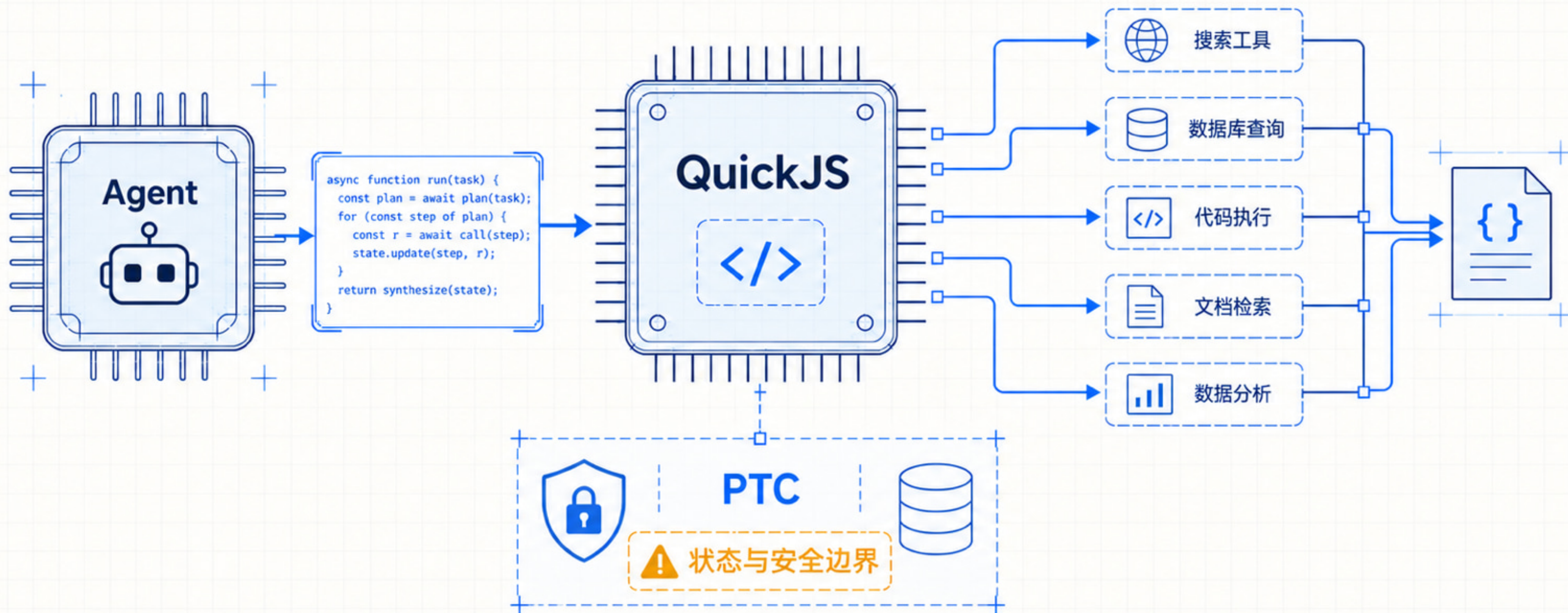


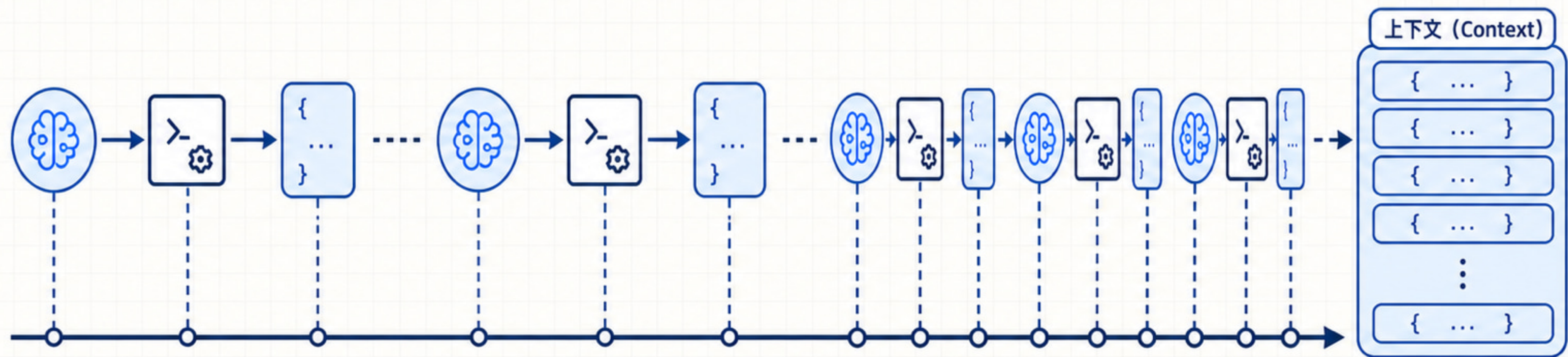
Deep Agents 实战

第 19 讲：Interpreters — 让 Agent 用代码编排工具与数据



顺序工具调用的问题，不在工具本身

中间结果越多，模型上下文越拥挤



⚠ 重复 × 80



模型决定



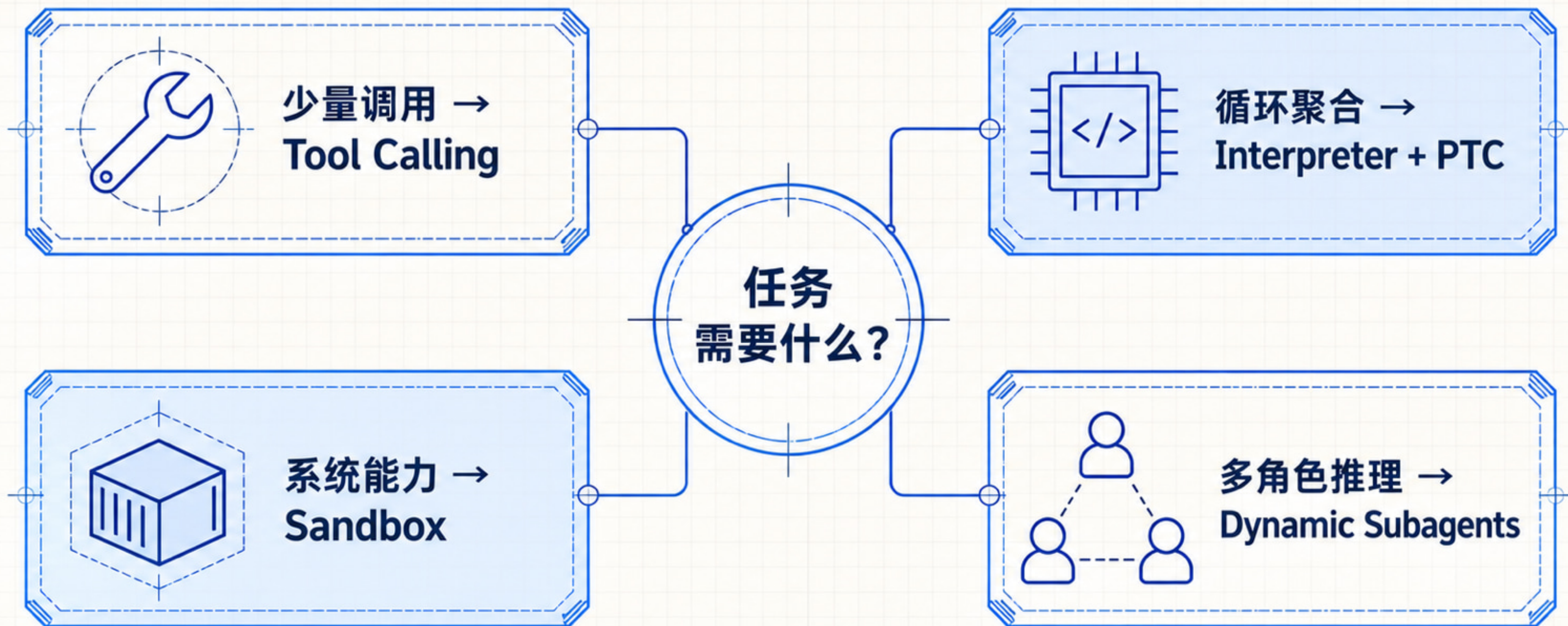
工具返回



结果进入上下文

先看任务形状，再决定执行机制

调用规模、系统能力和推理角色是三个分岔点



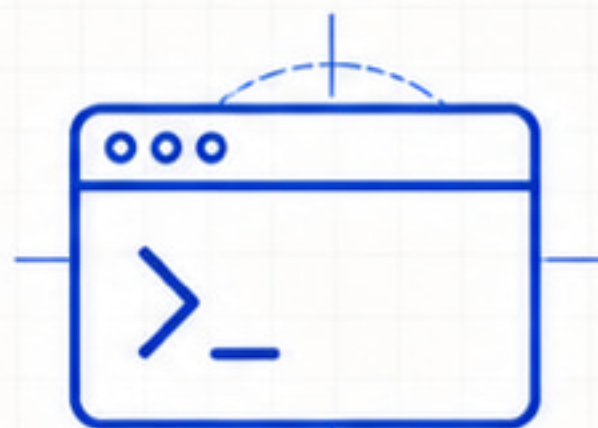
QuickJS 是引擎，langchain-quickjs 是 Python 桥

CodeInterpreterMiddleware 才是 Deep Agents 的接入点



三步把 Interpreter 接进 Agent

1 安装依赖



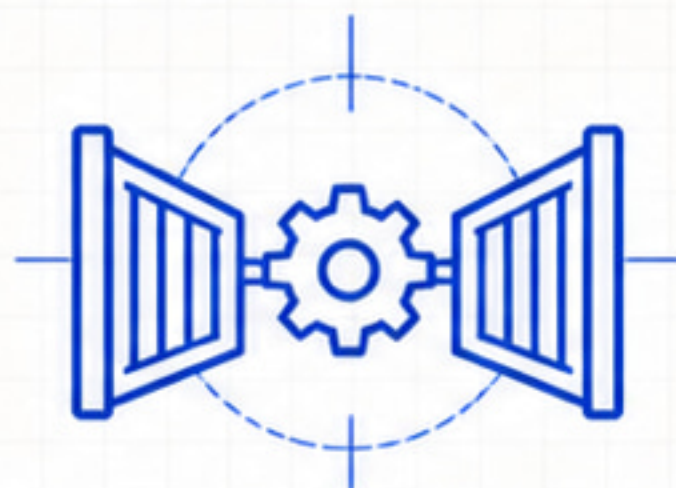
```
uv add "deepagents[quickjs]"
```

2 导入中间件



```
from langchain_quickjs import CodeInterpreterMiddleware
```

3 加入 middleware



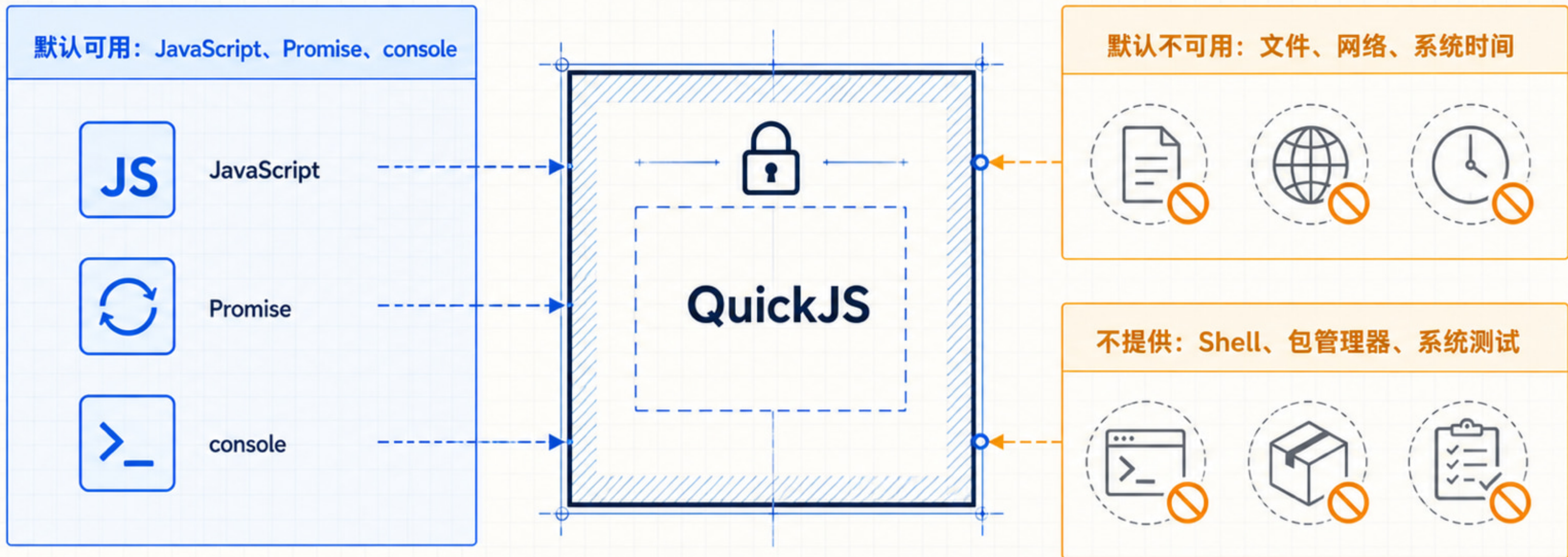
```
middleware=[CodeInterpreterMiddleware()]
```

```
await agent.ainvoke(...)
```

★ 推荐异步调用

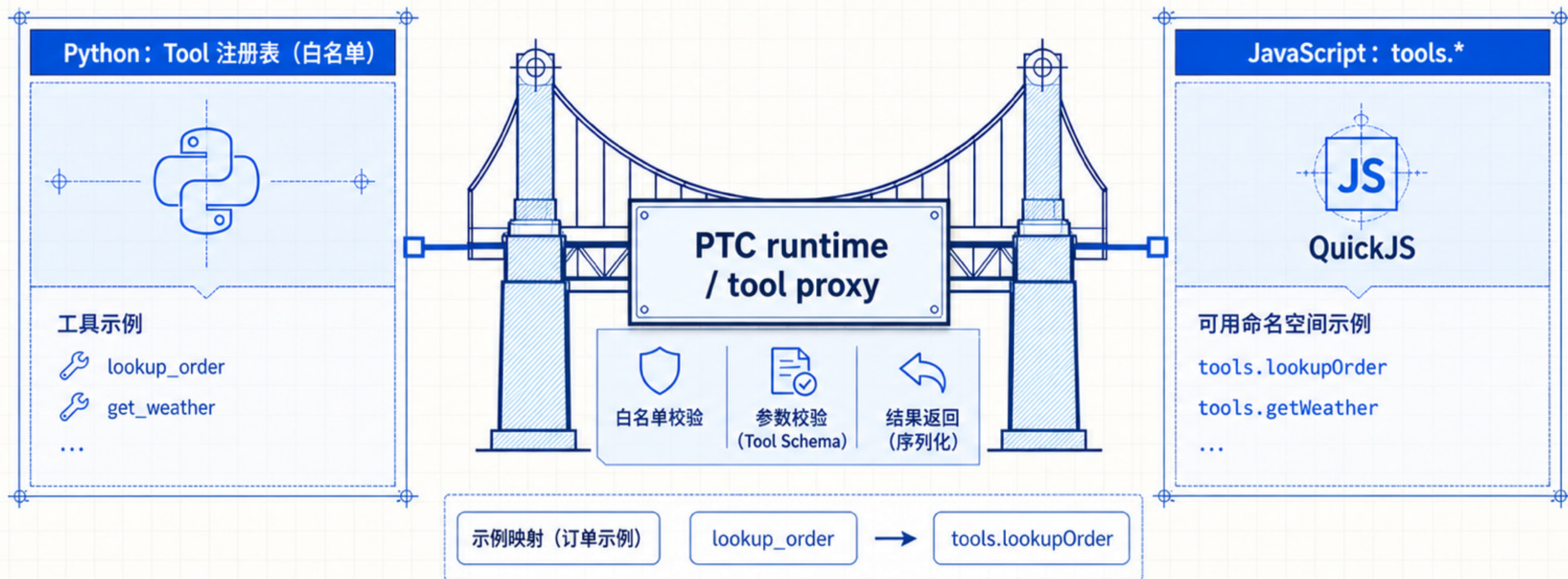
默认封闭，权限通过工具逐项打开

QuickJS 不是轻量版 Shell



PTC runtime 把白名单工具映射为 tools.*

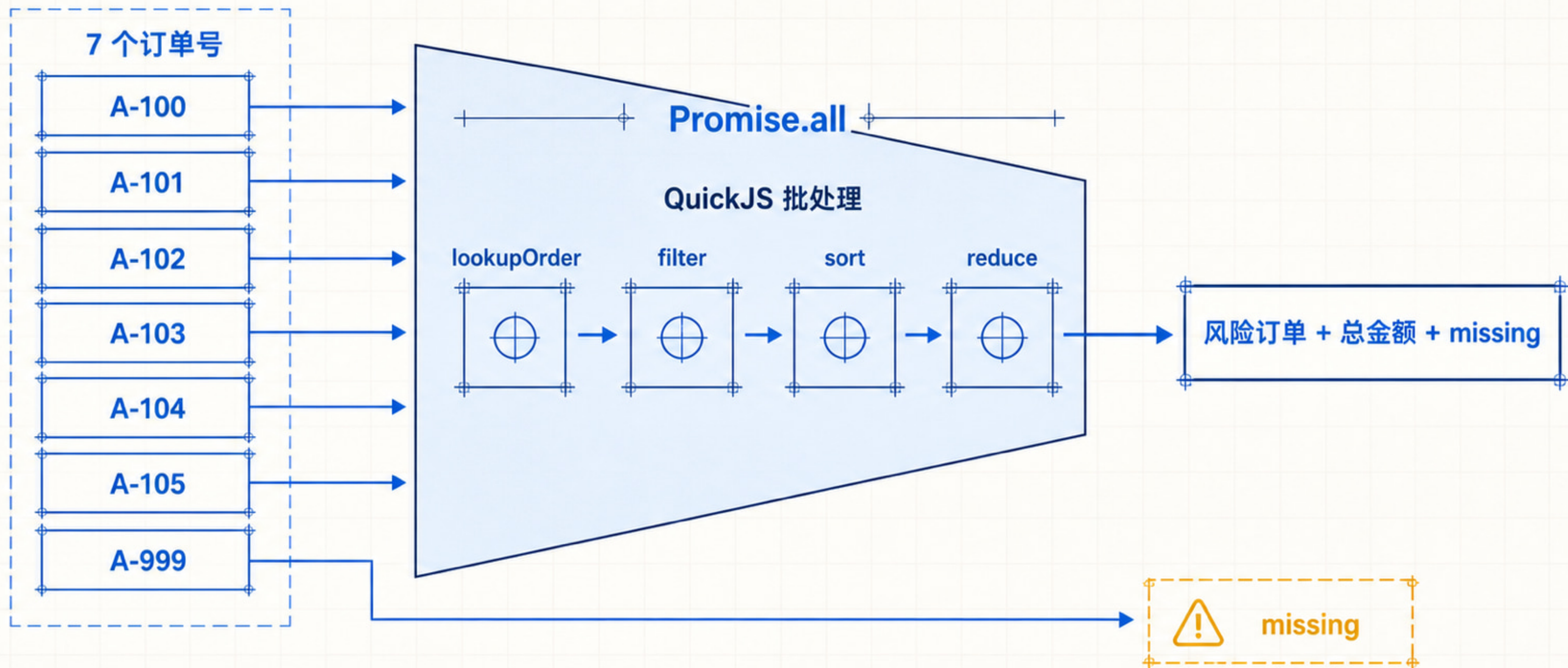
lookupOrder 只是订单示例，不是专用桥接函数



函数名转 camelCase, 参数仍遵循 Tool Schema

并行查询留在代码，模型只收汇总结果

Promise.all 覆盖全部输入，缺失项单独归类



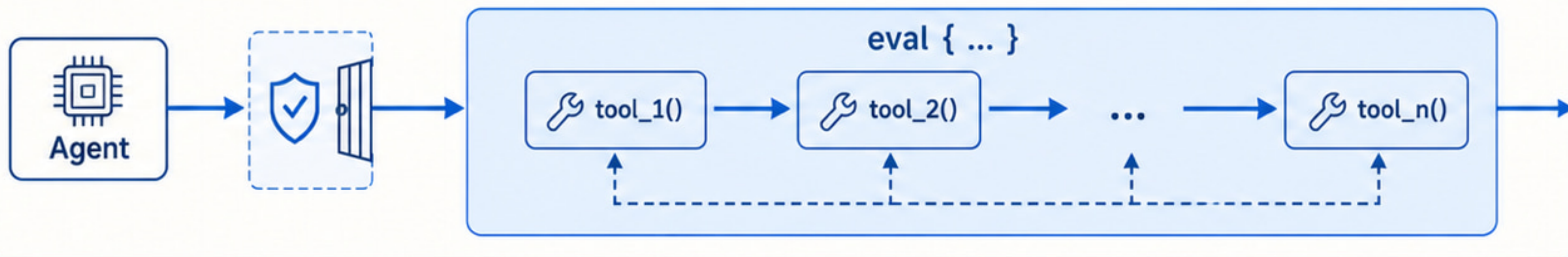
PTC 改变调用路径，也改变审批边界

interrupt_on 不会逐次覆盖 PTC 内部调用

普通 Tool Calling: 逐次审批



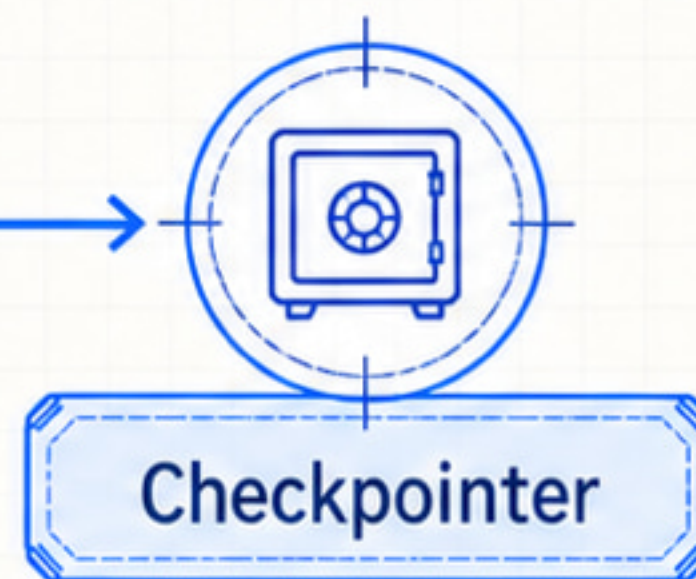
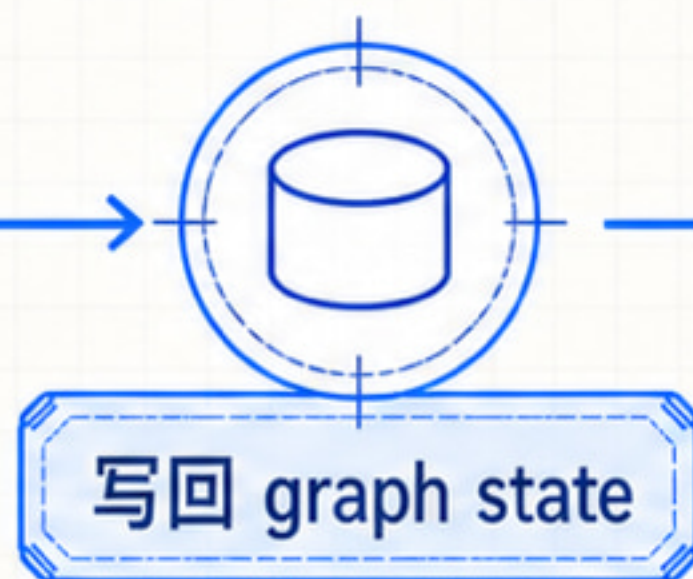
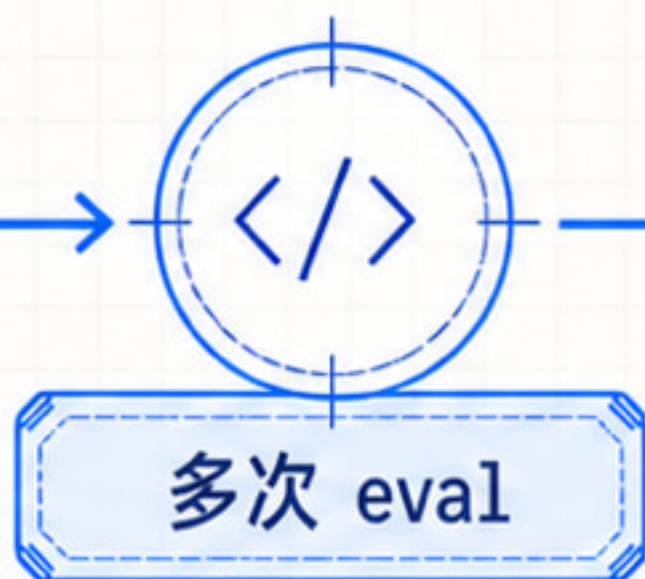
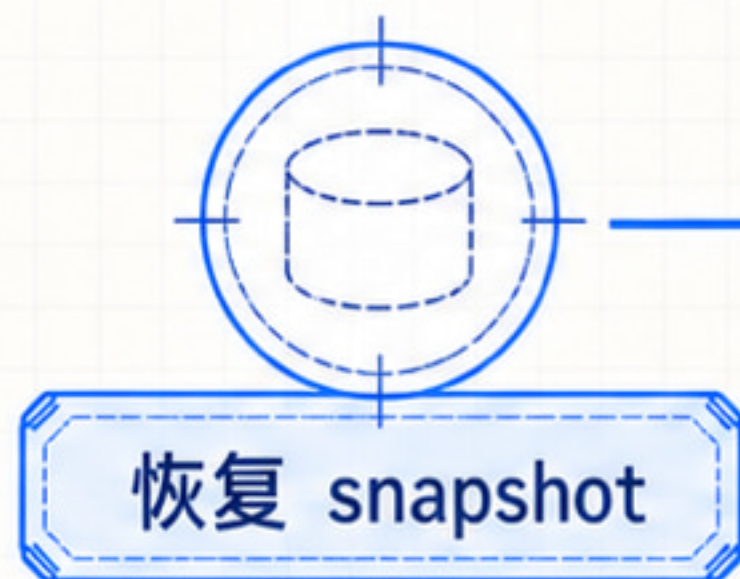
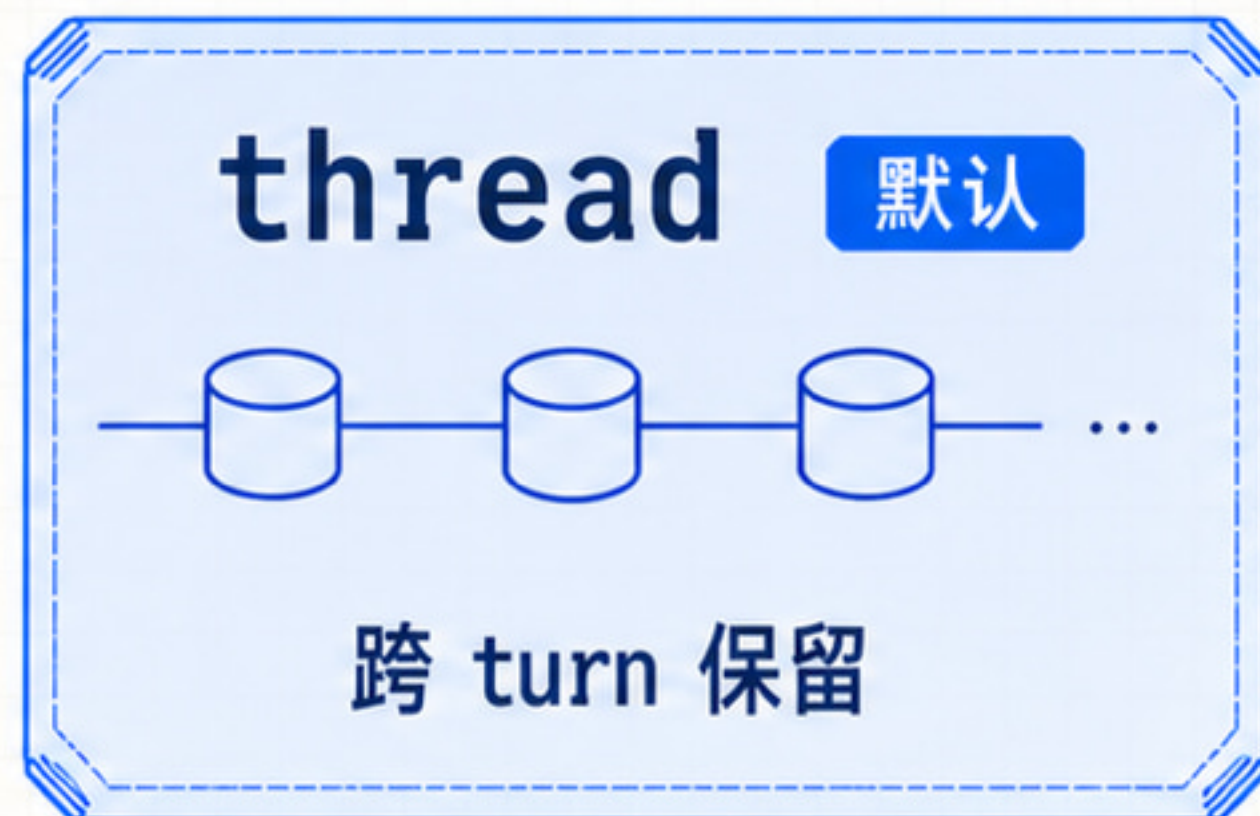
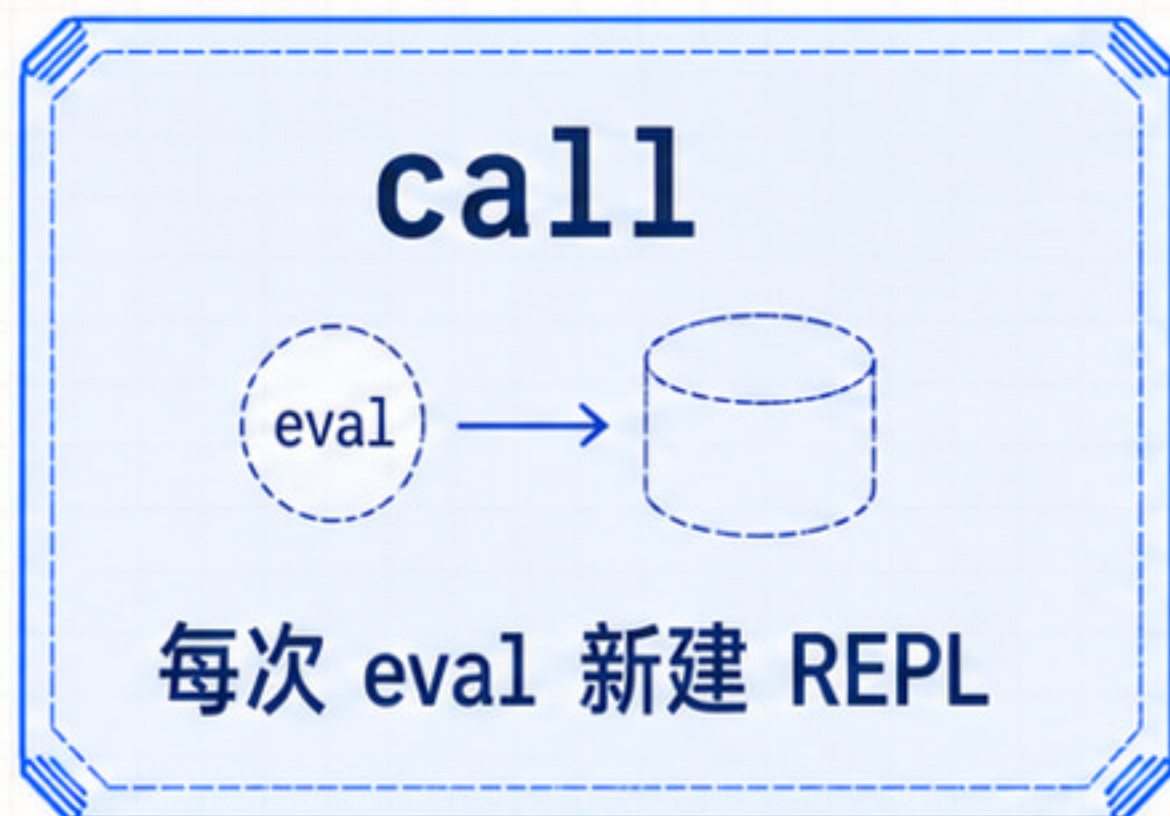
PTC: eval 内部批量执行



⚠
高副作用写操作
保留普通路径

mode 决定状态保留到哪里

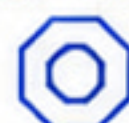
Checkpoint 只在 durable thread 或 time travel 时需要



 Snapshot 不会撤销 PTC 工具副作用

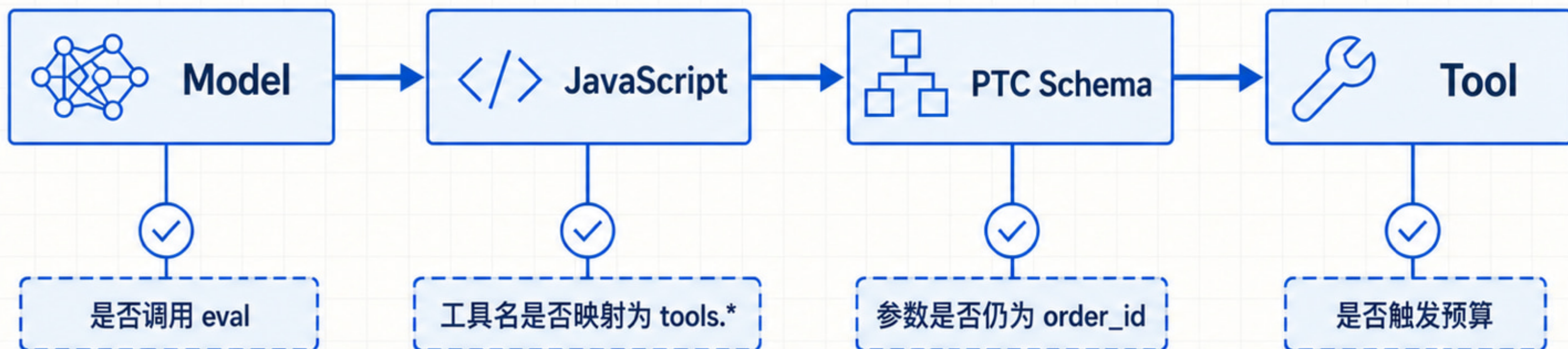
CodeInterpreterMiddleware 的十个配置项

先看默认值，再按风险收紧

 资源		memory_limit	=	64 MiB	
		timeout	=	5.0	
 能力		tool_name	=	"eval"	
		capture_console	=	True	
		max_result_chars	=	4000	
		ptc	=	None	
		max_ptc_calls	=	256	
		subagents	=	True	
 输出与状态		mode	=	"thread"	
		max_snapshot_bytes	=	None	
	CodeInterpreterMiddleware(timeout=3.0, mode="thread")				

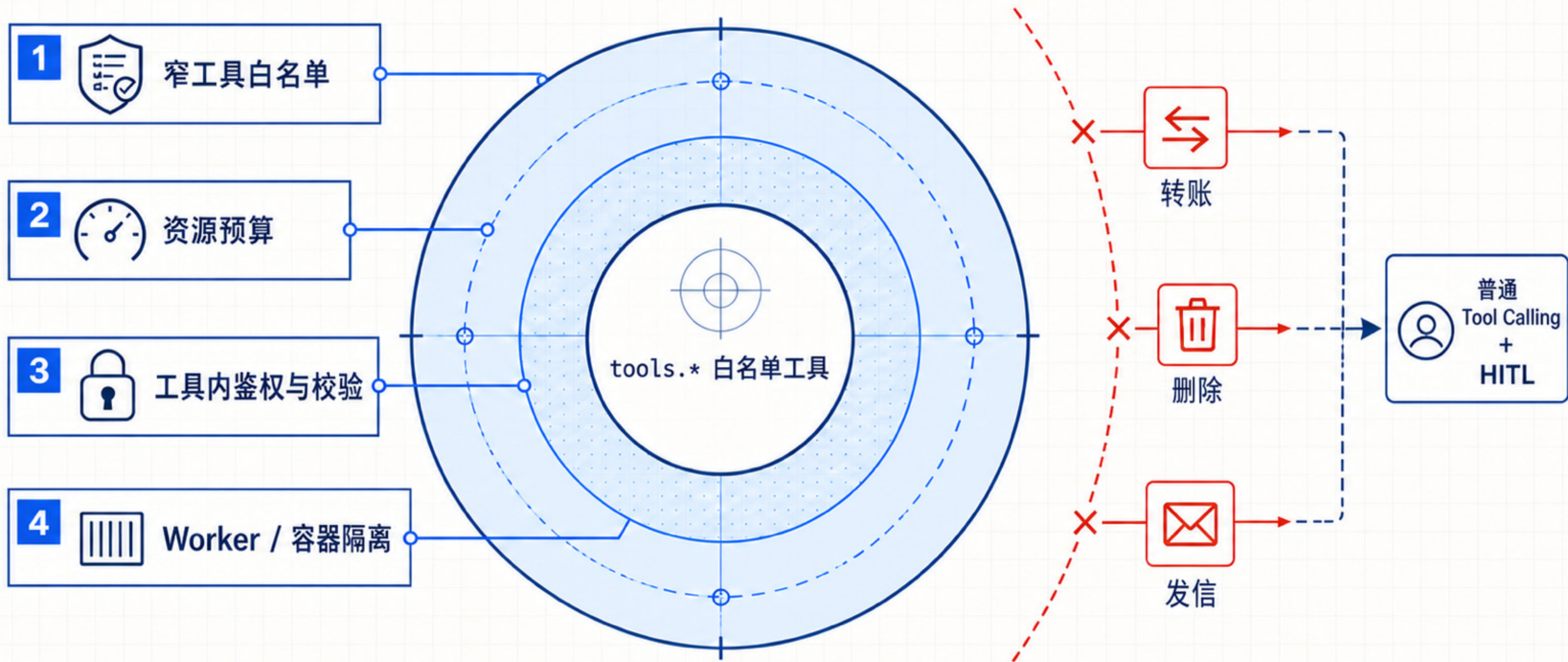
沿调用链排错，别只改 Prompt

Model → JavaScript → PTC Schema → Tool



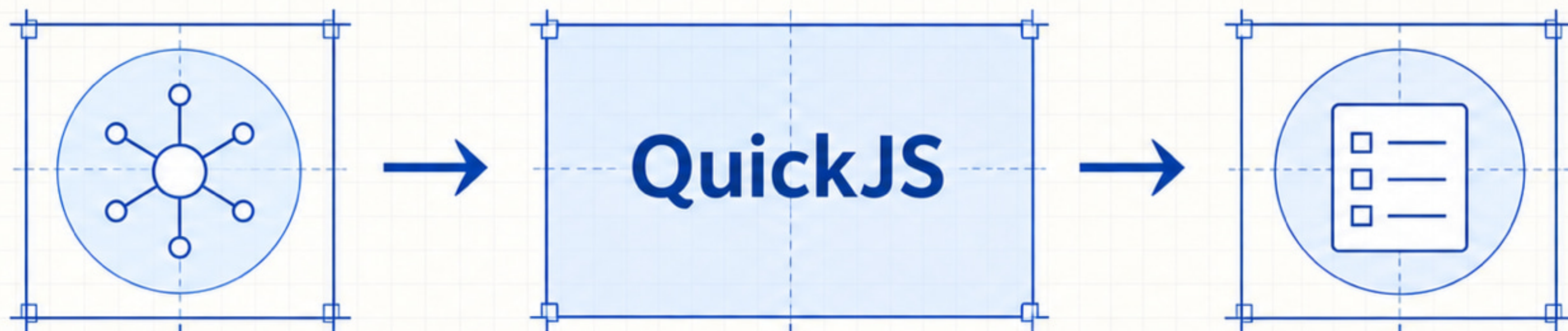
PTC 安全靠四层边界，不靠一个开关

工具能做什么，解释器代码就能做什么



把 Interpreter 用在确定性编排上

模型决定目标，代码负责循环、分支和聚合



任务形状匹配



PTC 白名单最小



状态范围够短



预算与审批清晰

四项全通过，才能上线执行。